



Universidade da Maia

Arquitetura e Design de Software

Gestor de Orçamento Pessoal

Júlio Araújo

Mestrado em Informática

07 Fevereiro 2026

1. Introdução

O Budget Tracker é uma aplicação web de gestão de finanças pessoais que permite ao utilizador registar, categorizar e visualizar as suas transações financeiras.

A aplicação organiza as transações em três tipos principais: despesas, receitas e investimentos, suportando tanto transações únicas como recorrentes (ex: salário mensal, subscrições, investimentos periódicos). O dashboard oferece uma visão clara do estado financeiro do utilizador através de gráficos interativos. O frontend é construído em React e o backend em Node.js com Express, utilizando um ORM para acesso à base de dados.

1.1 Objetivos

- Permitir o registo, edição e remoção de transações financeiras
- Categorizar transações por tipo (despesa, receita, investimento) e por categoria personalizável
- Suportar transações recorrentes (ex: salário mensal, renda, subscrições) com geração antecipada de ocorrências
- Apresentar um dashboard com resumo financeiro (totais, balanço, distribuição por categoria)
- Filtrar e pesquisar transações por tipo, categoria e período temporal

1.2 Funcionalidades Detalhadas

Gestão de Transações

- Criar transações com valor, descrição, data, tipo e categoria
- Editar e remover transações existentes
- Visualizar lista de transações com ordenação por data
- Filtrar por tipo (despesa/receita/investimento), categoria e intervalo de datas

Transações Recorrentes

Ao criar uma transação recorrente, o sistema gera automaticamente todas as ocorrências para o período de um ano. Esta abordagem simplifica a implementação e garante que todas as transações futuras estão imediatamente disponíveis na base de dados.

- Definir transações como recorrentes com frequência configurável (semanal, mensal, anual)
- Geração imediata de todas as ocorrências para 12 meses (ex: subscrição mensal gera 12 transações)
- Exemplos de uso: salário mensal (income), renda (expense), subscrições Netflix/Spotify (expense), investimento mensal em ETFs (investment)
- Dashboard filtra automaticamente para mostrar apenas transações até à data atual
- Possibilidade de cancelar ou editar todas as ocorrências futuras de uma subscrição de uma só vez (via `recurring_group_id`)

Gestão de Categorias

- Criar, editar e remover categorias personalizadas
- Associar categorias a tipos específicos (ex: “Alimentação” apenas para despesas)
- Categorias pré-definidas por defeito (Salário, Alimentação, Lazer, Ações, ETFs, etc.)

Dashboard e Visualizações

- Resumo do mês atual: total de receitas, despesas, investimentos e balanço
- Gráfico circular (pie chart) com distribuição de despesas por categoria
- Gráfico de barras ou linhas com evolução mensal (receitas vs despesas ao longo do tempo)
- Lista das últimas transações e próximas recorrências agendadas

2. Stack Tecnológica

Componente	Tecnologia	Justificação
Backend	Node.js + Express	Framework minimalista com vasta comunidade e documentação
Base de Dados	SQLite	Ficheiro local, sem necessidade de servidor de BD, ideal para aplicação single-user
ORM	Drizzle ORM	ORM leve com sintaxe próxima de SQL
Frontend	React + shadcn/ui	Arquitetura baseada em componentes
Gráficos	Recharts	Biblioteca de gráficos nativa para React, ideal para visualizações no dashboard
Controlo de Versões	Git + GitHub	Histórico de desenvolvimento e colaboração
Containerização	Docker + Docker Compose	Facilita a operacionalização

3. Arquitetura da Aplicação

A aplicação segue uma arquitetura em camadas, que promove a separação de responsabilidades e facilita a manutenção e evolução do sistema. A comunicação entre o frontend e o backend é feita através de uma API REST.

3.1 Descrição das Camadas

Camada de Apresentação (Frontend)

Responsável pela interface com o utilizador. Inclui o dashboard com resumo financeiro e gráficos (Recharts), formulários para registo de transações e a visualização de dados. Segue uma arquitetura baseada em componentes. Comunica com o backend exclusivamente através de chamadas HTTP à API REST.

Camada de API / Controladores (Routes)

Recebe os pedidos HTTP do frontend, valida os parâmetros de entrada e encaminha para a camada de serviços. Define os endpoints REST da aplicação e é responsável pelo formato das respostas HTTP.

Camada de Lógica de Negócio (Services)

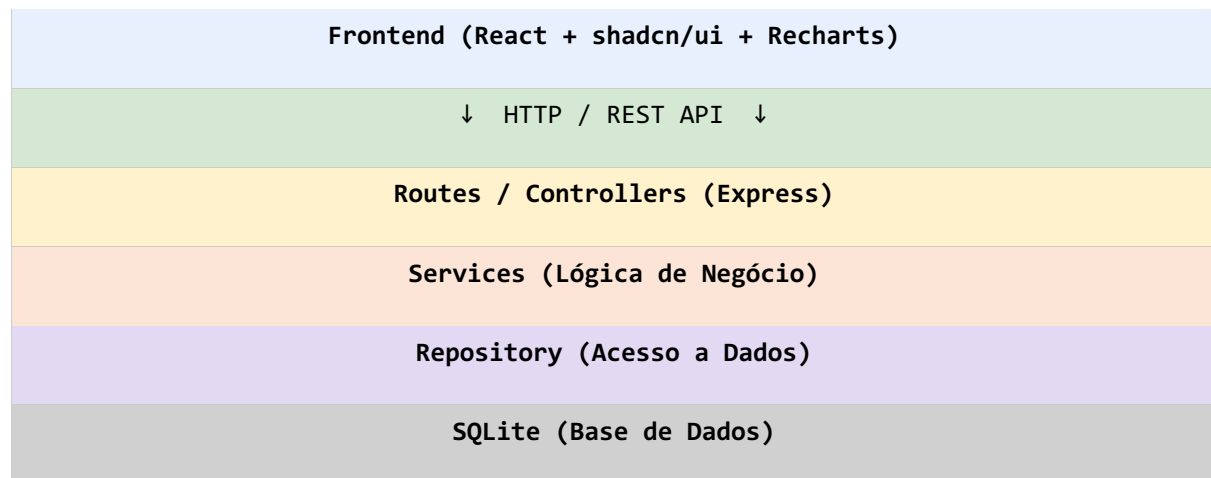
Contém as regras de negócio da aplicação, como o cálculo de totais, validações de domínio e agregação de dados para o dashboard. Esta camada é independente do framework web e da base de dados.

Camada de Acesso a Dados (Repository)

Responsável por todas as operações de leitura e escrita na base de dados. Utiliza o ORM como camada de abstração. Implementa o Repository Pattern, isolando os detalhes de persistência do resto da aplicação.

3.2 Diagrama de Camadas

Nota: Abaixo encontra-se uma representação textual da arquitetura em camadas. Um diagrama visual (UML) poderá ser incluído numa versão futura do documento.



3.3 Padrões Arquiteturais Utilizados

Padrão	Descrição e Aplicação
Layered Architecture	Separação em camadas com responsabilidades distintas, promovendo baixo acoplamento e alta coesão
MVC	Model-View-Controller: o modelo representa os dados, a view a interface, e o controller gere os pedidos
Repository Pattern	Abstração do acesso a dados através do Drizzle ORM, permitindo trocar a tecnologia de persistência sem afetar a lógica de negócio
REST	Comunicação cliente-servidor através de endpoints HTTP padronizados (GET, POST, PUT, DELETE)
Component-Based	Arquitetura do frontend baseada em componentes React reutilizáveis e independentes, promovendo modularidade

4. Modelo de Dados

A base de dados é composta por duas tabelas principais. O modelo foi desenhado para ser simples mas extensível, permitindo adicionar novas funcionalidades (como orçamentos ou metas) no futuro.

4.1 Tabela: categories

Armazena as categorias disponíveis para classificar transações (ex.: Alimentação, Salário, Ações).

Campo	Tipo	Restrições	Descrição
id	INTEGER	PK, AUTO	Identificador único da categoria
name	TEXT	NOT NULL	Nome da categoria (ex.: Alimentação, Transporte)
type	TEXT	NOT NULL	Tipo: expense, income ou investment

4.2 Tabela: transactions

Armazena todas as transações financeiras do utilizador.

Campo	Tipo	Restrições	Descrição
id	INTEGER	PK, AUTO	Identificador único da transação
type	TEXT	NOT NULL	Tipo: expense, income ou investment
amount	REAL	NOT NULL	Valor monetário da transação
description	TEXT		Descrição opcional da transação
date	TEXT	NOT NULL	Data da transação (formato ISO 8601)
category_id	INTEGER	FK	Referência à tabela categories
created_at	TEXT	DEFAULT NOW	Data e hora de criação do registo

recurring_group_id	INTEGER	NULL	Identificador do grupo de transações recorrentes (permite editar/cancelar todas de uma vez)
---------------------------	---------	------	---

4.3 Relacionamentos

categories → transactions (1:N): Cada categoria pode ter múltiplas transações associadas, mas cada transação pertence a apenas uma categoria. A foreign key **category_id** na tabela transactions referencia o **id** da tabela categories.

Agrupamento de recorrências: O campo **recurring_group_id** permite agrupar transações que pertencem à mesma subscrição ou recorrência. Todas as transações geradas a partir de uma mesma subscrição partilham o mesmo **recurring_group_id**, facilitando operações como cancelar ou editar a subscrição completa.

5. API REST — Endpoints

A comunicação entre o frontend e o backend é feita através de uma API RESTful. Abaixo encontram-se os principais endpoints previstos.

5.1 Transações

Método	Endpoint	Descrição
GET	/api/transactions	Listar transações (com filtros opcionais por tipo, categoria, data)
GET	/api/transactions/:id	Obter detalhe de uma transação específica
POST	/api/transactions	Criar nova transação
PUT	/api/transactions/:id	Atualizar uma transação existente
DELETE	/api/transactions/:id	Remover uma transação

5.2 Categorias

Método	Endpoint	Descrição
GET	/api/categories	Listar todas as categorias
POST	/api/categories	Criar nova categoria
PUT	/api/categories/:id	Atualizar uma categoria
DELETE	/api/categories/:id	Remover uma categoria

5.3 Recorrências

Método	Endpoint	Descrição
POST	/api/transactions/recurring	Criar transação recorrente (gera automaticamente todas as ocorrências para 12 meses)
GET	/api/transactions/recurring	Listar grupos de transações recorrentes (subscrições ativas)
DELETE	/api/transactions/recurring/:groupId	Cancelar subscrição: remove todas as transações futuras do grupo
PATCH	/api/transactions/recurring/:groupId	Editar subscrição: atualiza valor/descrição de todas as transações futuras

5.4 Dashboard

Método	Endpoint	Descrição
GET	/api/dashboard	Dados agregados: totais por tipo, balanço, distribuição por categoria

8. Melhorias Futuras

Esta secção documenta possíveis evoluções da aplicação que não serão implementadas na versão inicial para o efeito da unidade curricular, mas que representam melhorias arquiteturais ou funcionais relevantes.

Processamento Periódico de Recorrências (Cron Job)

A abordagem atual gera todas as transações recorrentes de uma só vez (12 meses). Uma alternativa seria implementar um sistema de processamento periódico: manter apenas um template da transação recorrente e utilizar um **cron job** que corre diariamente para gerar as transações quando a data de vencimento é atingida. Esta abordagem requer campos adicionais como **next_due_date** e **frequency**, bem como um serviço de agendamento (ex: node-cron). Embora mais complexa, permite recorrências infinitas.

Orçamentos por Categoria

Permitir ao utilizador definir limites de gasto mensais por categoria (ex: “300€ máximo em Alimentação”). O dashboard mostraria o progresso em relação ao orçamento definido, com alertas visuais quando o utilizador se aproxima ou ultrapassa o limite. Requer uma tabela adicional **budgets** com campos para categoria, valor limite e mês de referência.

Autenticação e Multi-utilizador

Adicionar sistema de autenticação (ex: JWT ou OAuth) para suportar múltiplos utilizadores, cada um com os seus próprios dados financeiros isolados. Requer uma tabela **users** e a adição de **user_id** como foreign key nas tabelas existentes.

Importação de Extratos Bancários

Permitir importar ficheiros CSV de extratos bancários para criação automática de transações, com sugestão automática de categorias baseada na descrição.